

HTML 마크업

웹 표준과 접근성을 고려한 시맨틱 마크업으로 구조화된 콘텐츠를 제공합니다.

시맨틱 태그

<header>, <nav>, <main>, <section>, <article>, <footer> 태그를 활용하여 의미있는 문서 구조를 구성했습니다.

웹 접근성

<h1>~<h2> 계층 구조, alt 속성, video 접근성 속성을 적용하여 모든 사용자가 콘텐츠에 접근할 수 있도록 했습니다.

SEO 최적화

페이지 구조를 제목 태그로 의미있게 구분하고, 링크와 이미지에 적절한 anchor text와 alt 속성을 사용해 검색 엔진 최적화를 구현했습니다.

```
dy>
header class="top">
  <div id="top-area">...
</div>
<div id="banner-area">  <!-- 3. 작가배너영역 -->
  <section class="banner-area inbox"></section>
  <section class="scroll-inbox"></section>
</div>
/header>
div id="main-area">  <!-- 4. 메인영역 -->
  <div id="spart-menu">  <!-- 스티커서브메뉴 -->
    <nav class="spart-menu inbox">
      <div id="spart-menu2" class="center-box"> </div>
    </nav>
  </div> <!-- 상단동그라미 -->
  <div class="main-area circle"></div>
  <main id="maincontent" class="main-area inbox">...
</main>
<div id="contact-area">  <!-- 5. 컨택영역 -->
  <div class="contact-area inbox">
    <div class="paddingbox">
      <h2 class="temp-tit blind">컨택영역</h2>
      <div class="main-area circle2"></div>
      <h2>Contact</h2>
      <p>For requests and inquiries, please contact us below</p>
      <div class="contact-box">
        <article class="name-box desc-box">...
        </article>
        <article class="email-box desc-box">...
        </article>
        <article class="details-box desc-box">...
        </article>
        <article class="transmit-box desc-box">...
        </article>
      </div>
    </div>
  </div>
```



styles.css

```
1  /* 1. 상단 영역 레이아웃 */
2  .top {
3      position: sticky;
4      top: 0;
5      left: 0;
6      height: 100vh;
7      height: 100dvh;
8      z-index: -1;
9  }
10
11 .top-area {
12     position: sticky;
13     background: url(../images/background.gif) no-repeat center top / cover;
14     padding: 0 0 2.7vw;
15     width: 100%;
16 }
17
18 .top-area .inbox {
19     position: fixed;
20     height: 125px;
21     margin-bottom: 40px;
22     padding: 24px 0 24px 0;
23 }
24
25 /* 2. 메인 메뉴 호버 애니메이션 */
26 .contact:hover img {
27     animation: menu-ani 2s linear forwards;
28 }
29
30 @keyframes menu-ani {
31     to {
32         padding: 0;
33         width: 100%;
34         height: 100%;
35     }
36 }
```

CSS 스타일링

주요 기술

- **레이아웃:** Flexbox와 Grid 12 System으로 유연한 구조 설계
- **포지셔닝:** sticky, fixed를 활용한 동적 UI 배치
- **애니메이션:** keyframes와 hover 효과로 생동감 있는 인터랙션 구현

방법론

- **모듈화:** 역할별로 파일을 분리하여 체계적 관리
- **컴포넌트 기반:** 독립적이고 재사용 가능한 UI 블록 설계
- **시맨틱 네이밍:** 의미있는 클래스명으로 코드 가독성 향상

반응형 웹 구현

다양한 디바이스 환경에서 최적의 사용자 경험을 제공하기 위해 4개의 주요 브레이크 포인트(1360px, 1200px, 1024px, 800px)를 설정하여 화면 크기별로 레이아웃을 세밀하게 조정했습니다.

01

미디어쿼리 적용

@media screen을 활용해 각 브레이크포인트별 스타일을 분리하고 구간별 최적화를 진행했습니다.

02

유연한 단위 시스템

vw, vh, %, rem 등 상대 단위를 활용하여 화면 크기에 따라 자연스럽게 조정되는 레이아웃을 구현했습니다.

03

플렉스박스 재정렬

모바일 메뉴에서 display: flex와 flex-direction을 활용해 구조를 동적으로 재배치했습니다.

04

반응형 이미지

aspect-ratio, object-fit: cover, width: 100%로 모든 해상도에서 최적의 이미지 표현을 구현했습니다.

테스트 전략: 뷰포트 기반 단위와 구간별 미디어쿼리 분리를 통해 각 환경에서의 정확한 테스트가 가능하도록 구조화했습니다.

```
/* 1360px 이하 미디어쿼리 시작 */
> @media screen and (max-width: 1360px) { ...
}
/* 1360px 이하 미디어쿼리 종료 */

/* 1200px 이하 미디어쿼리 시작 */
> @media screen and (max-width: 1200px) { ...
}
/* 1200px 이하 미디어쿼리 종료 */

/* 1024px 이하 미디어쿼리 시작 */
> @media screen and (max-width: 1024px) { ...
}
/* 1024px 이하 미디어쿼리 종료 */

/* 800px 이하 미디어쿼리 시작 */
@media screen and (max-width: 800px) {
  /* ////////////화면전체*/
  /* 모바일용 메뉴 */

  /* 메뉴그룹 변경 */
  .menu-group {
    height: 100%;
    display: flex;
    flex-direction: column;
    /* padding: 5vw; */
    background-color: white;
    justify-content: flex-start;
    align-items: flex-start;
    box-sizing: border-box;
    /* padding: 10vh 5vw; */
  }
}
```

JavaScript 구현 - 공통 부분

현재 페이지 상태에 따라 네비게이션 메뉴를 자동으로 활성화하는 인터랙티브 시스템을 구축했습니다.



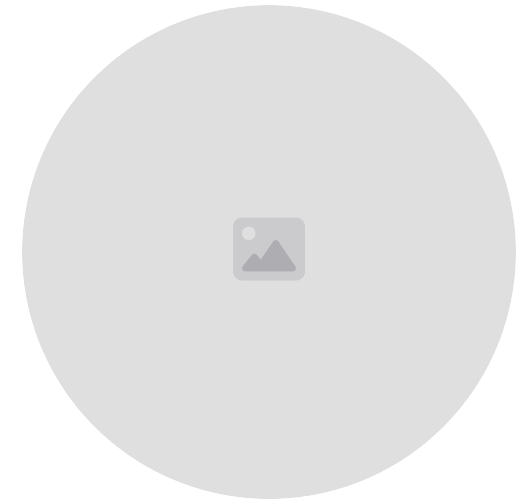
DOM 조작

querySelectorAll, classList 메서드를 활용
해 HTML 요소를 탐색하고 클래스를 제어하여
시각적 상태를 관리합니다.



URL API 활용

window.location을 통해 현재 페이지 정보와
해시를 가져와 메뉴 활성화 조건을 처리합니다.



Intersection Observer

사용자가 특정 섹션에 진입하거나 벗어날 때 메
뉴 상태를 자동으로 갱신하는 스크롤 기반 인터
랙션을 구현했습니다.

📌 **핵심 기능:** 페이지 그룹별 활성화 로직, 예외 처리(특정 작가명 배제), 해시 링크 처리, 실시간 섹션 감지로 사용자의 현재 위치를 명확하게 표시합니다.

JavaScript 구현 - 작품 페이지

페이지네이션, 작품 리스트, 모달 상세보기가 결합된 작품 아카이브 시스템입니다.

```
1 window.addEventListener("DOMContentLoaded", () => {
2   const wrap = document.querySelector(".wrap");
3   const pagingList = document.querySelectorAll(".dot");
4
5   const UNIT_NUM = 6;
6   const workTitle = [ ...
7
8   ];
9   // 페이지 초기값
10  let currentPage = 1;
11
12  // DOM 요소 캐시
13  const leftBtn = document.querySelector('.abtn.ab1');
14  const rightBtn = document.querySelector('.abtn.ab2');
15  const dotsContainer = document.querySelector('.paging-list .dots');
16
17  // totalPages 계산
18  const totalPages = Math.max(1, Math.ceil(workTitle.length / UNIT_NUM));
19
20  // 동적으로 dot이 필요하면 렌더링 (선택; 이미 HTML에 dot이 있으면 이 블록은 건너뛰어도 됩니다)
21  function renderDotsIfNeeded() {
22    const existingDots = dotsContainer.querySelectorAll('.dot');
23    if (existingDots.length === totalPages) return; // 이미 맞춤
24
25    dotsContainer.innerHTML = '';
26    for (let i = 1; i <= totalPages; i++) {
27      const span = document.createElement('span');
28      span.className = 'dot';
29      span.dataset.val = String(i);
30      dotsContainer.appendChild(span);
31    }
32  }
33}
```

```
38 // dot 스타일 업데이트 (활성화 표시)
39 function updateDots(page) {
40   const dots = dotsContainer.querySelectorAll('.dot');
41   dots.forEach(dot => {
42     const val = Number(dot.dataset.val);
43     if (val === page) {
44       dot.style.backgroundColor = "#666666";
45       dot.classList.add("active");
46     } else {
47       dot.style.backgroundColor = '';
48       dot.classList.remove("active");
49     }
50   });
51
52   // 좌/우 버튼 활성화/비활성화(선택) 처리: 끝이면 비활성화 시키기
53   if (leftBtn) leftBtn.classList.toggle('disabled', page <= 1, );
54   if (rightBtn) rightBtn.classList.toggle('disabled', page >= totalPages);
55 }
56
57 // 페이지 이동 함수 (MakeWorkList 호출 + 상태 업데이트)
58 function gotoPage(page) {
59   // 범위 검사
60   if (page < 1) page = 1;
61   if (page > totalPages) page = totalPages;
62
63   currentPage = page;
64   // 실제 리스트 렌더링
65   MakeWorkList(currentPage);
66
67   // dot 상태 업데이트
68   updateDots(currentPage);
69 }
```

```
70
71 // 왼쪽/오른쪽 버튼 이벤트 바인딩
72 if (leftBtn) {
73   leftBtn.addEventListener('click', () => {
74     if (currentPage > 1) gotoPage(currentPage - 1);
75   });
76 }
77 if (rightBtn) {
78   rightBtn.addEventListener('click', () => {
79     if (currentPage < totalPages) gotoPage(currentPage + 1);
80   });
81 }
82
83 // dots 클릭 바인딩 (delegation 또는 개별 바인딩)
84 function bindDotClicks() {
85   // 이벤트 위임: dotsContainer에 클릭될거
86   dotsContainer.addEventListener('click', (e) => {
87     const dot = e.target.closest('.dot');
88     if (!dot) return;
89     const page = parseInt(dot.dataset.val, 10);
90     if (Number.isNaN(page)) return;
91     gotoPage(page);
92   });
93 }
94
95 // 초기화 (페이지 수에 따라 dot 렌더링/바인딩 후 첫 렌더 호출)
96 renderDotsIfNeeded();
97 bindDotClicks();
98 gotoPage(1); // 초기 페이지 렌더링
99
```

주요 구현 기능

1. **작품 리스트 출력:** 페이지당 6개 작품을 동적으로 표시
2. **Dot 페이지네이션:** 작품 개수에 따라 자동 생성되는 페이지 점
3. **페이지 이동:** 좌우 버튼 또는 dot 클릭으로 전환

JavaScript 구현 - 작품 페이지

주요 구현 기능

4. 모달 상세보기: 작품 클릭 시 이미지, 제목, 제작연도, 기법 정보 표시

5. 스크롤 제어: 모달 오픈 시 body 스크롤 방지

핵심 기술

데이터 관리: workTitle과 workInfo 배열로 구조적 데이터 관리

최적화: 조건부 렌더링과 배열 탐색 최적화로 효율적 검색 구현

반응형 UX: display와 overflow 제어로 상태별 시각적 전환

```
269 // 4. 모달 기능
270 const mWindow = document.getElementById("modal");
271 Windsurf: Refactor | Explain | Generate JSDoc | X
272 function openModal(e) {
273     let idx = e.currentTarget.parentElement.getAttribute('data-num');
274     console.log(idx);
275     // 배열.some((v)=>{if(조건){실행}})
276     // some은 배열을 순회하면서 조건에 맞으면
277     // 실행문을 실행하고 return true를 쓰면 끝마친다!
278     workInfo.some(v=>{
279         if(v.idx == idx){
280             console.log(v.title);
281             mWindow.querySelector('h3').textContent = v.title;
282             mWindow.querySelector('.date').textContent = v.date;
283             mWindow.querySelector('.technique').textContent = v.technique;
284             mWindow.querySelector('.dimensions').textContent = v.dimensions;
285             return true;
286         }
287     });
288     console.log(v.title);
289
290 })
291 mWindow.style.display = "block";
292 mWindow.querySelector('.modal-img img').src = `./images/paint${idx}.jpg`;
293 document.body.style.overflow = "hidden"; // 바디스크롤막기효과
294 }
295
296
```

JavaScript 구현 - 층 페이지

층별 버튼 클릭 시 해당 층의 지도 이미지와 설명을 전환하는 인터랙티브 층 안내 시스템입니다.

- 라벨 기반 페이지 이동

data-url 속성을 읽어 해당 페이지로 이동하고 라디오 버튼 체크 상태를 동기화합니다.

- 자동 페이지 인식

현재 URL과 label 속성을 비교하여 페이지 로드 시 자동으로 해당 층을 선택합니다.

- 층별 상태 관리

선택층은 활성 이미지로, 위층은 숨김으로, 아래층은 흐림 처리하여 시각적 계층을 구현했습니다.

데이터 속성 활용

data-url, data-floor 속성으로 층 정보와 페이지 정보를 HTML에 저장하여 JS에서 효율적으로 조회합니다.

배열 기반 관리

floorsOrder와 mapPos 배열로 층 순서와 지도 이동 위치를 체계적으로 관리하여 유지보수성을 향상시켰습니다.

floor_map_compactjs

```
1 "keyword">const DEFAULT_FLOOR = "5F";
2 "keyword">const floorsOrder = ["5F", "4F", "3F", "2F", "1F", "B1F"];
3 "keyword">const mapPos = [18, 8, 0, -8, -16, -9];
4
5 "keyword">const floorItems = document.querySelectorAll(".floortxt");
6 "keyword">const mapWrap = document.querySelector(".mapwrap");
7 "keyword">const maps = document.querySelectorAll(".mapwrap > div");
8
9 "keyword">let currentOpenFloor = null;
10
11 "keyword">const floorImages = {
12   "5F": { default: "./images/gallery1.png", active: "./images/gallery1.png" },
13   "4F": { default: "./images/4f_down.png", active: "./images/gallery2.png" },
14   "3F": { default: "./images/3f_down.png", active: "./images/gallery3.png" },
15   "2F": { default: "./images/2f_down.png", active: "./images/gallery4.png" },
16   "1F": { default: "./images/1f_down.png", active: "./images/gallery5.png" },
17   "B1F": { default: "./images/gallery6.png", active: "./images/gallery6.png" }
18 };
19
20 "keyword">function updateMaps(activeFloor) {
21   "keyword">const index = floorsOrder.indexOf(activeFloor);
22   mapWrap.style.translate = `0 ${mapPos[index]}px`;
23
24   maps.forEach((m) => {
25     "keyword">const i = floorsOrder.indexOf(m.dataset.floor);
26     "keyword">const floor = m.dataset.floor;
27     "keyword">const imgEl = m.querySelector("img");
28
29     "keyword">if (i < index) {
30       imgEl.src = floorImages[floor].default;
31       Object.assign(m.style, { opacity: 0, visibility: "hidden", zIndex: 0, pointerEvents: "none" });
32     } "keyword">else "keyword">if (i === index) {
33       imgEl.src = floorImages[floor].active;
34       Object.assign(m.style, { opacity: 1, filter: "none", zIndex: 99, visibility: "visible", pointerEvents: "auto" });
35     } "keyword">else {
36       imgEl.src = floorImages[floor].default;
37       Object.assign(m.style, { opacity: 0.3, filter: "blur(2px)", visibility: "visible", zIndex: 0, pointerEvents: "none" });
38     }
39   });
40 }
41
42 "keyword">function setActive(floor) {
43   floorItems.forEach((li) => {
44     li.classList.toggle("active", li.dataset.floor === floor);
45   });
46
47   "keyword">if (floor === currentOpenFloor) "keyword">return;
48   updateMaps(floor);
49   currentOpenFloor = floor;
50 }
51
52 floorItems.forEach((li) => {
53   li.addEventListener("click", () => setActive(li.dataset.floor));
54 });
55
56 setActive(DEFAULT_FLOOR);
```


JavaScript 구현 - 전시 페이지

화면 스크롤 중 특정 요소(.bottom-area)가 뷰포트에 진입하는 순간을 감지하여 <main> 요소에 .end 클래스를 추가하거나 제거하는 시각적 상태 전환 기능입니다.



실시간 스크롤 감지

scroll 이벤트로 window.scrollToY와 요소의 getBoundingClientRect().top을 지속적으로 비교합니다.



뷰포트 진입 판단

요소의 top 좌표가 window.innerHeight보다 작아지면 화면에 들어온 것으로 판단하여 .end 클래스를 추가합니다.



CSS 트리거 역할

.end 클래스가 배경색 전환, 페이드인 효과, 고정 헤더 해제 등의 스타일 변화를 촉발합니다.

활용 기술 스택

- 순수 JavaScript (ES6)
- document.querySelector() DOM 탐색
- getBoundingClientRect() 위치 계산
- addEventListener('scroll') 이벤트 제어
- classList API 상태 전환

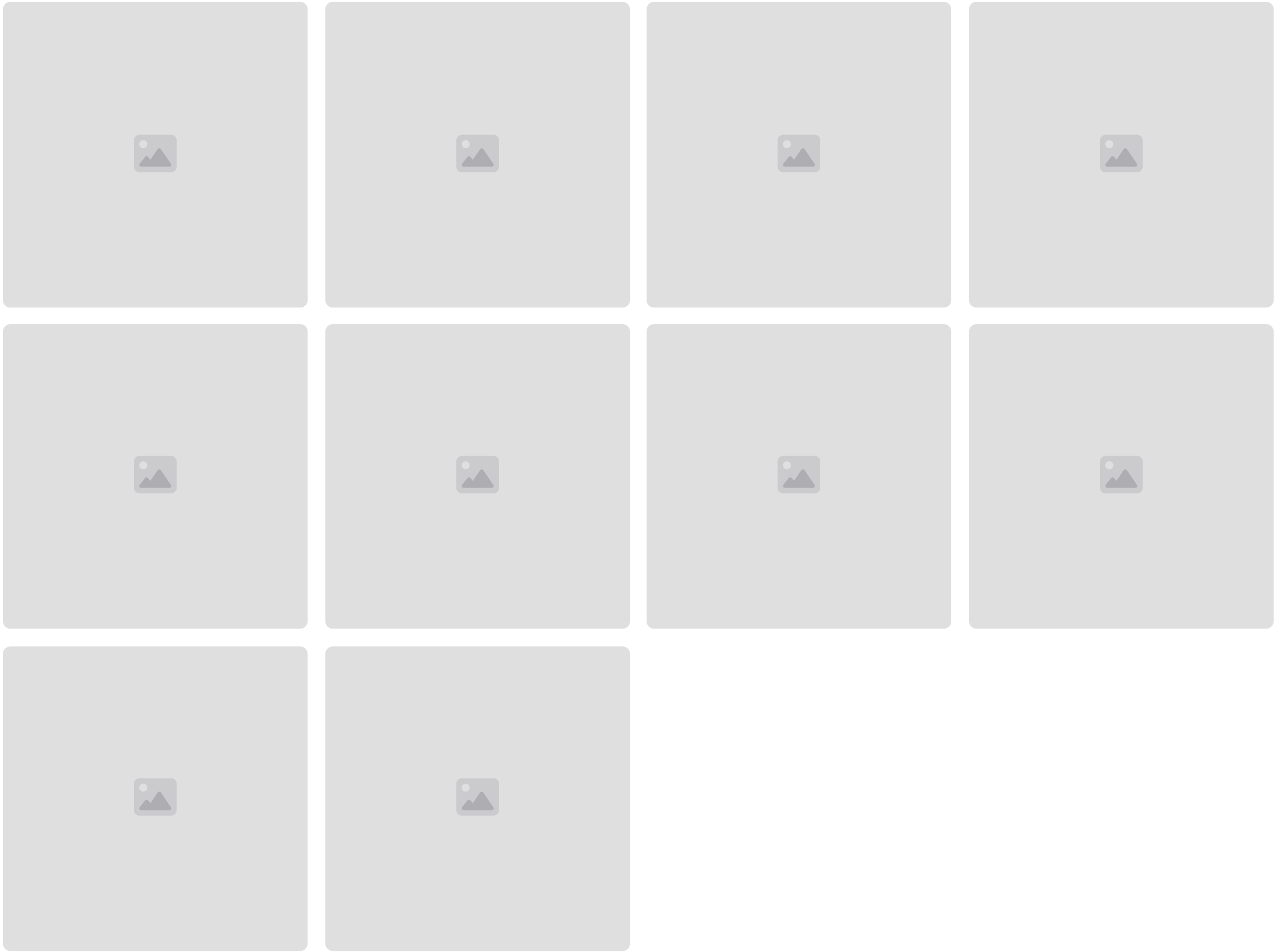
```
Windsurf: Refactor | Explain | X
const getBCR = (el) => el.getBoundingClientRect().top;

// 타겟박스
const target = document.querySelector('.bottom-area');
// 변경박스
const chgBox = document.querySelector('main');
// 윈도우 높이값 (기준값)
let winH = window.innerHeight;

window.addEventListener('scroll', () => {
  let scTop = window.scrollToY;
  let tgPos = getBCR(target);
  console.log('스크롤이동중', scTop, tgPos);
  if(tgPos < winH) {
    chgBox.classList.add('end');
  }
  else {
    chgBox.classList.remove('end');
  }
});
```


프로젝트 결과물

최종적으로 10개 페이지의 완전한 반응형 웹사이트를 구현했습니다. 각 페이지는 웹 표준과 접근성을 준수하며, 다양한 JavaScript 인터랙션으로 향상된 사용자 경험을 제공합니다.



10

페이지 퍼블리싱

완전한 반응형 웹 디자인 구현

4

브레이크포인트

다양한 디바이스 최적화

 개선 방향

미디어쿼리 보완: 더 세밀한 브레이크포인트 추가로 다양한 디바이스 대응

새로운 페이지 생성: 추가 콘텐츠 페이지 개발 및 기능 확장

감사합니다

프로젝트 발표 끝

2025 웹퍼블리셔 과정 반응형 웹사이트 프레젠테이션을 마치겠습니다.